HDOC 39

Name = arayana khare

Sap id = 590042432

Experiment 1: Variadic Function to Calculate Weighted Average

Aim: To write a C program using a variadic function `weightedAverage(int count, ...)` to calculate the weighted average and validate weights.

C Program

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...)
{
    va_list args;
    double value, weight;
    double sum = 0, totalWeight = 0;

    va_start(args, count);

    for (int i = 0; i < count; i++)
    {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0)
        {
            printf("Error: Negative weight!\n");
            va_end(args);
            return -1;
        }

        sum += value * weight;
        totalWeight += weight;
    }
```

```
    va_end(args);

    if (totalWeight == 0)
    {
        printf("Error: Total weight is zero!\n");
        return -1;
    }

    return sum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(3,
                             80.0, 0.3,
                             75.0, 0.2,
                             90.0, 0.5);

    if (result != -1)
        printf("Weighted Average = %.2f\n", result);

    return 0;
}
```

Algorithm

1. Start the program.

2. Define a variadic function using `stdarg.h`.

3. Read each value and its corresponding weight.

4. Check that every weight is non-negative.

5. Calculate the sum of value × weight and the total weight.

6. If the total weight is zero, display an error; otherwise, return the weighted average.

7. Display the result and stop.

Output

```
Weighted Average = 84.00
```

Experiment 2: Command-Line Arguments and Statistical Calculations

Aim: To write a C program that accepts integer command-line arguments, ignores invalid inputs, sorts valid integers, and calculates minimum, maximum, average, median, and second-largest distinct value.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

int isInteger(char *s)
{
    char *end;
    long n;

    if (*s == '\0')
        return 0;

    n = strtol(s, &end, 10);

    return *end == '\0' &&
           end != s &&
           n >= INT_MIN && n <= INT_MAX;
}

void sort(int a[], int n)
{
```

```
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
}

int main(int argc, char *argv[])
{
    int a[argc], n = 0, i;
    int min, max, second = 0, found = 0;
    long long sum = 0;
    double avg, median;

    for (i = 1; i < argc; i++)
    {
        if (isInteger(argv[i]))
        {
            a[n] = atoi(argv[i]);
            sum += a[n];
            n++;
        }
        else
            printf("Invalid argument ignored: %s\n",
                   argv[i]);
    }

    if (n == 0)
    {
        printf("No valid integers found.\n");
        return 0;
    }

    sort(a, n);

    printf("Valid integers in ascending order: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    min = a[0];
```

```
    max = a[n - 1];
    avg = (double)sum / n;

    if (n % 2 == 0)
        median = ((double)a[n / 2 - 1] +
                  a[n / 2]) / 2;
    else
        median = a[n / 2];

    for (i = n - 2; i >= 0; i--)
    {
        if (a[i] != max)
        {
            second = a[i];
            found = 1;
            break;
        }
    }

    printf("\nMinimum: %d", min);
    printf("\nMaximum: %d", max);
    printf("\nAverage: %.2f", avg);
    printf("\nMedian: %.2f", median);

    if (found)
        printf("\nSecond-largest distinct value: %d",
               second);
    else
        printf("\nNo second-largest distinct value");

    printf("\n");
    return 0;
}
```

Algorithm

1. Start the program and accept command-line arguments.

2. Validate each argument using a user-defined function.

3. Ignore invalid arguments and store valid integers in an array.

4. Sort the integers in ascending order.

5. Calculate minimum, maximum, average, and median.

6. Find the second-largest distinct value.

7. Display all results and stop.

Compilation and Execution

Save the file as `program.c`.

```
gcc program.c -o program
./program 12 5 abc 8 12 20 3
```

Output

```
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

Important Viva Points

- `argc` stores the number of command-line arguments, including the program name.

- `argv` stores the arguments as strings.

- `strtol()` converts a string into a number and helps validate the complete input.

- `sort()` arranges numbers in ascending order.

- The median of an even number of elements is the average of the two middle elements.

- The second-largest distinct value excludes duplicates of the maximum.